

Logistic Regression

In this file, we will go through the mathematical details of Logistic Regression so that we can implement it. This will include a derivation of the Logistic Regression algorithm (specifically the gradient descent update rule) starting from the Bernoulli distribution. (However, we will leave certain details out, specifically the derivation of the sigmoid function, as this requires a deeper discussion with GLMs and the exponential family, however the intuition should suffice.)

Let's start with the Bernoulli distribution:

$$P(Y) = \phi^y (1-\phi)^{1-y} \text{ where } y \in \{0, 1\}$$

We want our hypothesis function to approximate ϕ , which is the probability that $y=1$. We can rewrite the Bernoulli distribution so that it is parameterized by θ and conditional on x :

$$P(Y|x; \theta) = h_{\theta}(x)^y (1-h_{\theta}(x))^{1-y} \text{ where } y \in \{0, 1\} \text{ and } h_{\theta}(x) \in [0, 1]$$

more specifically, $h_{\theta}(x)$ is our hypothesis function, and is defined as

$$h_{\theta}(x) = \sigma(\theta^T x) = \frac{1}{1+e^{-\theta^T x}}$$

here, $\sigma(z) = \frac{1}{1+e^{-z}}$, $\sigma: \mathbb{R} \mapsto [0, 1]$ is the sigmoid function, which takes any real number, and maps it to be between 0 and 1. Note that we are taking an inner product between our parameters (θ) and the data vector (x), so we will use the convention that $x_0 = 1$ to allow for an intercept term, and therefore, $\theta \in \mathbb{R}^{n+1}$, $x \in \mathbb{R}^{n+1}$ where n is the number of features.

Given a set of training examples $(x^{(1)}, y^{(1)}), \dots, (x^{(m)}, y^{(m)})$, we can define the Likelihood function as

$$\mathcal{L}(\theta) = P(\vec{Y}|\vec{X}; \theta) = \prod_{i=1}^m P(y^{(i)}|x^{(i)}; \theta) = \prod_{i=1}^m h_{\theta}(x^{(i)})^{y^{(i)}} (1-h_{\theta}(x^{(i)}))^{1-y^{(i)}}$$

To simplify this product, we can consider taking the logarithm (since it is strictly increasing), arriving at the "log likelihood": $\log \mathcal{L}(\theta) = \log \mathcal{L}(\theta)$

$$\log \mathcal{L}(\theta) = \log \left(\prod_{i=1}^m h_{\theta}(x^{(i)})^{y^{(i)}} (1-h_{\theta}(x^{(i)}))^{1-y^{(i)}} \right) = \sum_{i=1}^m y^{(i)} \log(h_{\theta}(x^{(i)})) + (1-y^{(i)}) \log(1-h_{\theta}(x^{(i)}))$$

In order to find our parameters θ , we want to maximize the likelihood (or log likelihood), a technique known as maximum likelihood estimation (MLE). Before we do this, we will make two changes to follow the standard convention, we will take the negative, arriving at the loss function, and divide by m to average our loss. Note that finding the maximum of the log likelihood is equivalent to finding the minimum of the negative log likelihood.

Let $J(\theta)$ denote our loss function:

(Note that this loss function is also referred to as Binary Cross-Entropy loss)

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m y^{(i)} \log(h_{\theta}(x^{(i)})) + (1-y^{(i)}) \log(1-h_{\theta}(x^{(i)}))$$

To find the parameters that minimize our loss function, we will use an iterative algorithm

known as gradient descent. Let's compute the gradient:

$$\begin{aligned} \frac{\partial J(\theta)}{\partial \theta_j} &= \frac{\partial}{\partial \theta_j} \left(-\frac{1}{m} \sum_{i=1}^m y^{(i)} \log(\sigma(\theta^T x^{(i)})) + (1-y^{(i)}) \log(1-\sigma(\theta^T x^{(i)})) \right) \\ &= -\frac{1}{m} \sum_{i=1}^m \frac{y^{(i)} \sigma(\theta^T x^{(i)}) (1-\sigma(\theta^T x^{(i)})) x_j^{(i)}}{\sigma(\theta^T x^{(i)})} + \frac{(y^{(i)}-1) (\sigma(\theta^T x^{(i)})) (1-\sigma(\theta^T x^{(i)})) x_j^{(i)}}{(1-\sigma(\theta^T x^{(i)}))} \end{aligned}$$

(Note that we used the property that $\sigma'(z) = \sigma(z)(1-\sigma(z))$)

$$\begin{aligned} &= -\frac{1}{m} \sum_{i=1}^m y^{(i)} (1-\sigma(\theta^T x^{(i)})) x_j^{(i)} + (y^{(i)}-1) \sigma(\theta^T x^{(i)}) x_j^{(i)} \\ &= -\frac{1}{m} \sum_{i=1}^m y^{(i)} x_j^{(i)} - y^{(i)} \sigma(\theta^T x^{(i)}) x_j^{(i)} + y^{(i)} \sigma(\theta^T x^{(i)}) x_j^{(i)} - \sigma(\theta^T x^{(i)}) x_j^{(i)} \\ &= \frac{1}{m} \sum_{i=1}^m (\sigma(\theta^T x^{(i)}) - y^{(i)}) x_j^{(i)} \end{aligned}$$

$$\nabla_{\theta} J(\theta) = \frac{1}{m} X^T (\sigma(X\theta) - Y)$$

where $X = \begin{bmatrix} x^{(1)T} \\ \vdots \\ x^{(m)T} \end{bmatrix}$ is the matrix of training examples where each row is an individual example, and $Y = \begin{bmatrix} y^{(1)} \\ \vdots \\ y^{(m)} \end{bmatrix}$ is the vector of labels. Moreover:

we can now arrive at the gradient descent update rule:

$$\theta := \theta - \alpha \left(\frac{1}{m} X^T (\sigma(X\theta) - Y) \right)$$

This will be the key equation that we implement so that our model can learn and develop a good, linear decision boundary (linear as we use $\theta^T x$). Note that this is batch gradient descent as we use the batch of examples to compute the gradient. (α is a hyperparameter that corresponds to the learning rate, typically set around 0.01).